CS683

Programming Assignment 2

Amey Dhoke 26M0795 | **Parthsinh Thakor** 26M0796 | **Deva Narayana B** 26M0797 | **Sumit Chavan** 26M0799

15 October 2026

Experimental setup

Parameter	Value
Simulator	ChampSim
Branch predictor	Hashed perceptron
Cores	1
Warmup instructions	25,000,000
Simulation instructions	25,000,000
Traces	Trace1, Trace2, Trace3

Every run used these instruction counts; only the binary and the trace changed. For example, SPP at the L1D with dynamic I-RRIP at the L2C on Trace3:

```
./bin/hashed_perceptron-no-spp-no-no-no-no-no-lru-lru-lru-dyn_irrip-lru-lru-lru-lru-1core-baseline \
    -warmup_instructions 25000000 \
    -simulation_instructions 25000000 \
    -traces traces/Trace3.gz
```

1 Data prefetching

1.1 Stride Pattern Prefetcher (SPP)

1.1.1 Results

Trace	Configuration	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage (%)
Trace1	No prefetching	0.232345	1.0000×	10.678	–	–
	IP-stride	0.232917	1.0025×	10.118	52.81	5.25
	SPP	0.234150	1.0078×	9.725	53.98	8.92
Trace2	No prefetching	0.251158	1.0000×	13.538	–	–
	IP-stride	0.250303	0.9966×	13.440	43.46	0.72
	SPP	0.252755	1.0064×	13.214	53.77	2.39
Trace3	No prefetching	0.382065	1.0000×	3.403	–	–
	IP-stride	0.382722	1.0017×	3.318	16.01	2.50
	SPP	0.388587	1.0171×	2.953	40.11	13.23

Table: L1D prefetchers with LRU at the L2C. Speedup and coverage are relative to no prefetching.

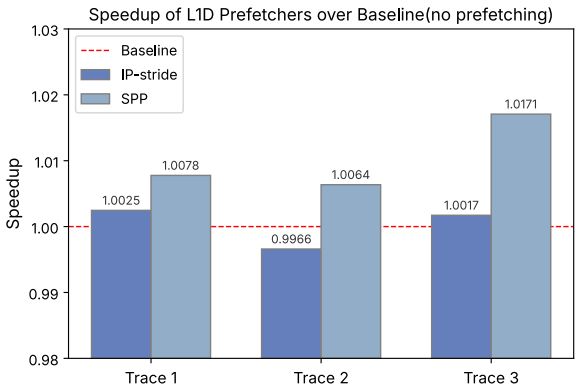


Figure 1: Speedup of IP-stride and SPP over no prefetching.

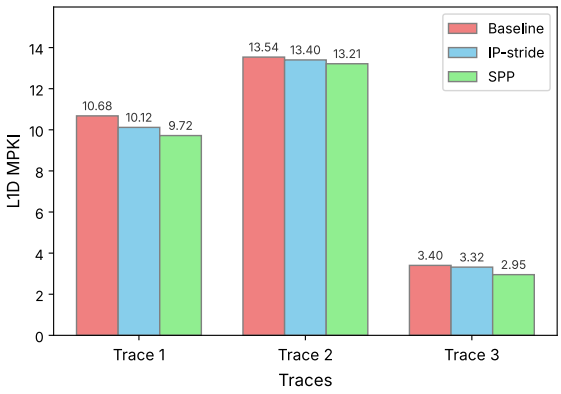


Figure 2: L1D load MPKI.

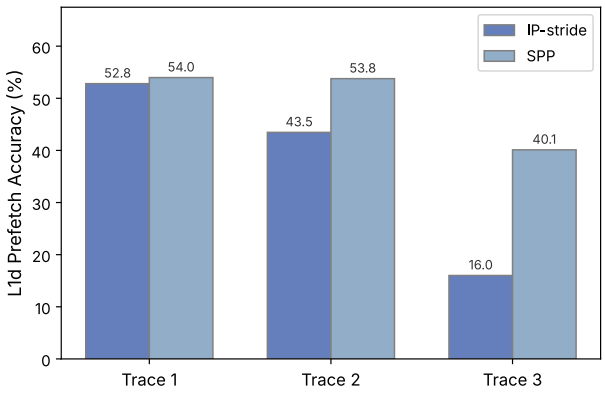


Figure 3: L1D prefetch accuracy.

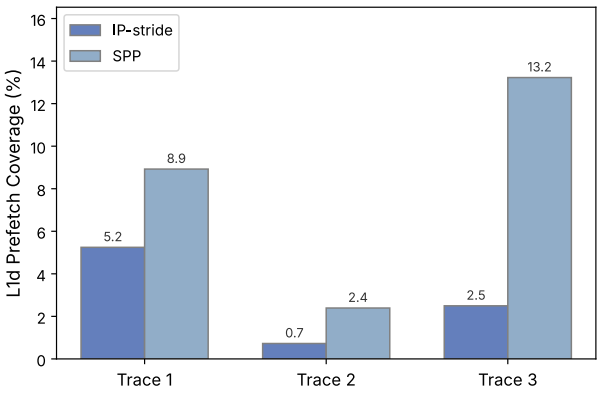


Figure 4: L1D prefetch coverage.

1.1.2 Storage overhead

Structure	Entries	Bits per entry	Total bits
IP table	64	42	2,688
Pattern table	64	9	576
Recent prefetch buffer	32	17	544
Throttling registers	4	–	26
Total(bits)			**3,834**
Total(bytes)			**479.25**

Table: SPP storage per structure.

Entry	Field	Bits	Description
IP table	valid	1	Is the entry legal?
	IP tag	10	To tell IPs apart
	page tag	10	To detect page change
	last offset	6	Which block within the 4 KB page?
	last stride	7	What was the last stride?
	confidence	2	How frequent is it?
	signature	6	Last stride hashed with low 3 bits of the previous one
Pattern table	next stride	7	Same range as the last stride
	confidence	2	How frequent is it?
Prefetch buffer	valid	1	cleared when used, so each prefetch counts once

	block tag	16	low 16 bits of the prefetched block address

Table: Bit-level breakdown of every SPP entry and register.

1.1.3 Reasoning

We chose degree values ranging from 1-4, the reason behind which is total MSHR entries are 16, so we wanted to keep some entries free for loads as well.
The accuracy thresholds that we chose are 25%, 50%, and 75% and the reason we chose these is that it divides our degrees neatly into 4 quartiles.

2 Instruction-conscious L2 replacement

2.A I-RRIP

We changed insertion, promotion, and eviction to reduce L2 instruction-load MPKI. Instruction misses can delay fetching, so keeping useful instruction lines in L2 can help performance. We also wanted to avoid removing frequently used data just to make space for instructions.

Insertion

We insert instruction lines at RRPV 2 and data lines at RRPV 3. This gives a new instruction line more protection than a new data line. We chose 2 instead of 0 because a newly inserted instruction has not yet shown reuse. Giving it maximum protection immediately could keep an unused instruction in the cache at the expense of useful data. Inserting data at 3 makes it easier to remove data with little reuse, although useful data may sometimes be evicted before its next access.

Promotion

On an instruction hit, we set its RRPV to 0. On a data hit, we decrease its RRPV by one, stopping at 0. This gives reused instructions immediate protection, while data earns protection through repeated hits. Promoting both directly to 0 would give them equal protection after one hit. Leaving data unpromoted would be too aggressive because even frequently accessed data would remain easy to evict.

Eviction

We first look for a data line at RRPV 3. If none exists and an instruction line is at RRPV 3, we prefer evicting a data line at RRPV 2. This gives the instruction line an extra chance to survive. If no data line has RRPV 2 or 3, we evict the instruction line at RRPV 3. This prevents instruction preference from overriding the protection given to data at RRPV 0 or 1. We leave aging unchanged: when no line has RRPV 3, we increment all RRPVs and repeat the search.

2.A.1 Results

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr. load MPKI	L2 data load MPKI
Trace1	LRU	0.232345	1.0000×	17.6956	10.3377	7.3580
	RRIP	0.232231	0.9995×	17.6387	10.5618	7.0769
	I-RRIP	0.237234	1.0210×	18.6026	9.0753	9.5274
Trace2	LRU	0.251158	1.0000×	27.3568	18.8533	8.5035
	RRIP	0.251806	1.0026×	25.7458	17.7813	7.9645
	I-RRIP	0.266050	1.0593×	18.6132	7.4791	11.1340
Trace3	LRU	0.382065	1.0000×	3.9542	1.1241	2.8302
	RRIP	0.381739	0.9991×	4.0395	1.1970	2.8424
	I-RRIP	0.385034	1.0078×	3.9938	0.7920	3.2018

Table: L2C replacement policies, no L1D prefetcher. Speedup is relative to LRU.

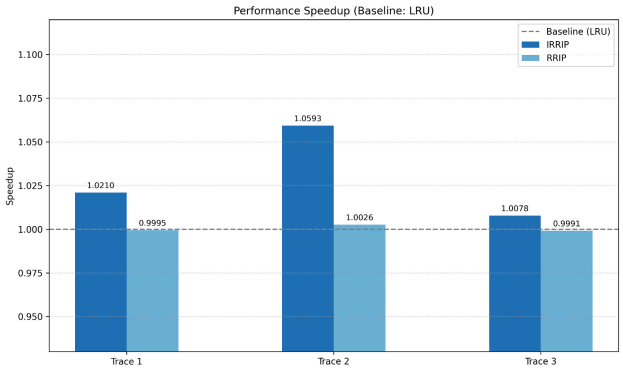


Figure 5: Speedup over LRU.

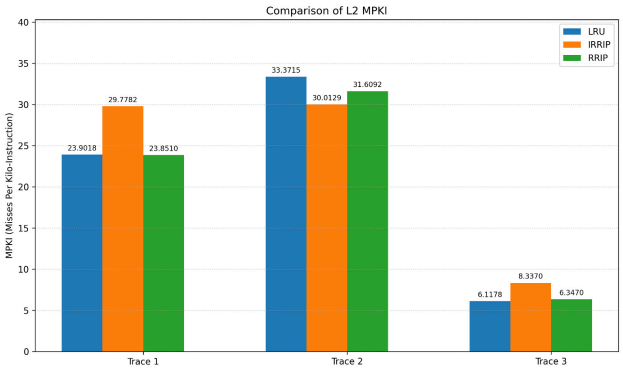


Figure 6: Total L2C MPKI (all access types).

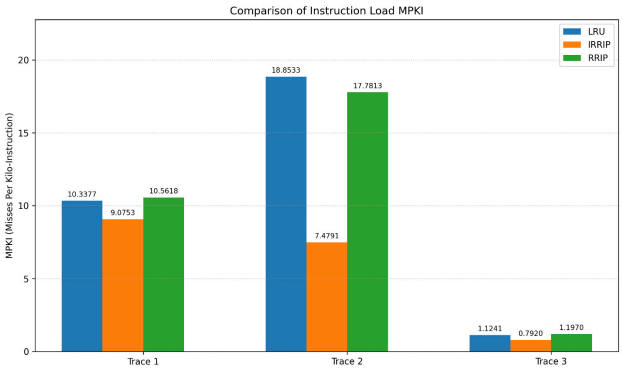


Figure 7: L2C instruction load MPKI.

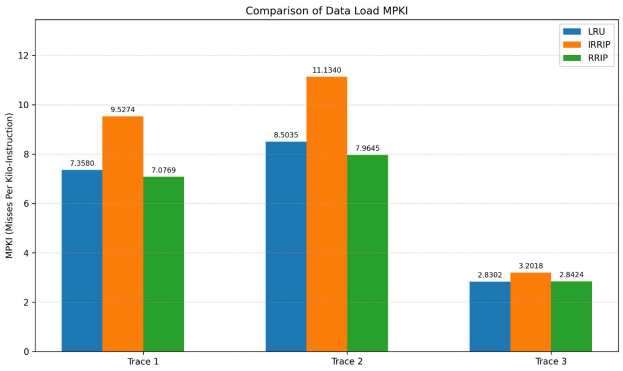


Figure 8: L2C data load MPKI.

We chose this combination because it favors instructions at each replacement decision without ignoring data reuse.

The results support this choice. Instruction-load MPKI decreases in all three traces, with a 37.39% reduction based on the geometric mean of MPKI ratios relative to LRU. Despite the increase in data-load MPKI, the geometric mean IPC speedup is 1.02914×, giving a 2.91% improvement over LRU. The policy therefore meets the instruction-miss reduction goal and improves performance on the tested workloads. Comparing other combinations would be needed to establish whether it is the best possible design.

2.B Dynamic I-RRIP

2.B.1 Sensitivity study

We tried five threshold values: 0.15, 0.20, 0.25, 0.30, and 0.35. We ran all three traces for each threshold and compared the geometric mean of IPC and the MPKI metrics.

A lower threshold makes the policy switch to instruction prioritization more easily. A higher threshold requires instruction misses to account for a larger share of the misses observed in the leader sets.

We chose 0.25 because it gave the highest geometric mean IPC, along with the lowest load and data MPKI. The lowest total MPKI occurred at 0.20, while the lowest instruction MPKI occurred at 0.30. At 0.25, instruction MPKI was close to its lowest value, and execution performance was the best among the values we tested.

The results were quite close across all five thresholds, but 0.25 gave us the best combination of IPC and load MPKI.

Threshold	0.15	0.20	0.25	0.30	0.35
IPC	0.289312	0.289311	**0.289317**	0.289281	0.289278
Total MPKI	19.5564	**19.5323**	19.5435	19.5635	19.5624
Load MPKI	11.2170	11.2164	**11.2122**	11.2210	11.2209
Data MPKI	6.9615	6.9577	**6.9537**	6.9634	6.9600
Instruction MPKI	3.8571	3.8553	3.8545	**3.8542**	3.8625

Table: Dynamic I-RRIP for five threshold values: geometric mean over the three traces of IPC and L2C MPKI. Bold marks the best value in each row.

2.B.2 Results

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr. load MPKI	L2 data load MPKI
Trace1	LRU	0.232345	1.0000×	17.6956	10.3377	7.3580
	RRIP	0.232231	0.9995×	17.6387	10.5618	7.0769
	Dynamic I-RRIP	0.237118	1.0205×	18.5877	9.0762	9.5115
Trace2	LRU	0.251158	1.0000×	27.3568	18.8533	8.5035
	RRIP	0.251806	1.0026×	25.7458	17.7813	7.9645
	Dynamic I-RRIP	0.265242	1.0561×	19.0203	7.9493	11.0710
Trace3	LRU	0.382065	1.0000×	3.9542	1.1241	2.8302
	RRIP	0.381739	0.9991×	4.0395	1.1970	2.8424
	Dynamic I-RRIP	0.385049	1.0078×	3.9869	0.7937	3.1932

Table: Dynamic I-RRIP (threshold 0.25) against LRU and RRIP, no L1D prefetcher. Speedup is relative to LRU.

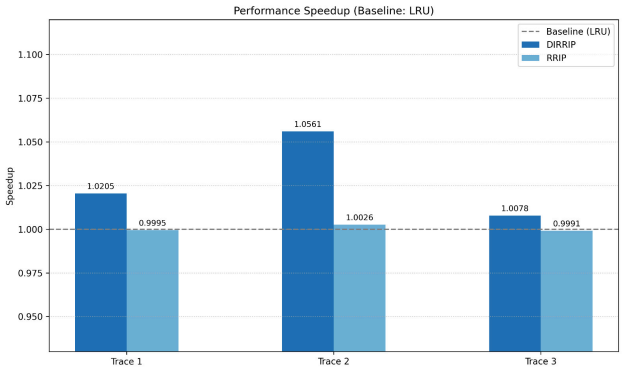


Figure 9: Speedup over LRU.

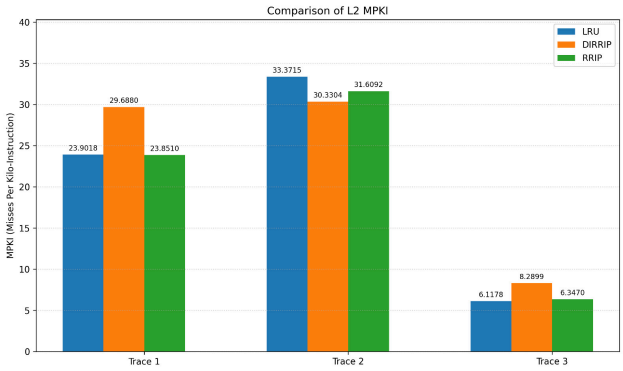


Figure 10: Total L2C MPKI (all access types).

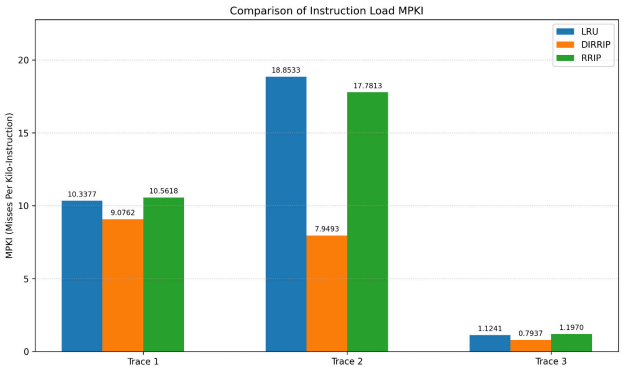


Figure 11: L2C instruction load MPKI.

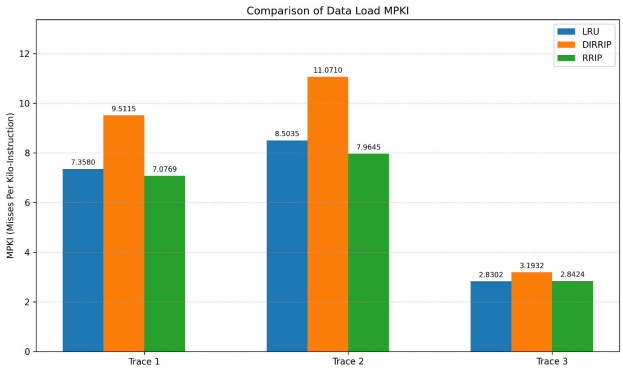


Figure 12: L2C data load MPKI.

2.B.3 Leader sets

We use a few leader sets to collect feedback and decide which policy the remaining sets should follow. Our L2 cache has 1024 sets. We randomly picked five positions, 1, 29, 77, 191, and 222, and use these positions in each group of 256 sets through `set % 256`. This gives us 20 leader sets and 1004 follower sets. There is no special reason behind the individual positions.

We kept the number of leaders small so that most sets can switch between RRIP and I-RRIP. The leaders make up about 1.95% of the cache sets. Choosing more leaders would give us more observations, but it would also leave fewer sets available to use instruction prioritization.

Leader policy

The leader sets always use regular RRIP. We track their instruction and non-instruction accesses and misses separately. Since their policy does not change, we can continue observing how regular RRIP handles the workload even when the followers use I-RRIP. Only leader-set accesses update these counters.

Decision logic

We use the following ratio to choose the follower policy:

$$\text{Instruction miss ratio} = \frac{\text{Instruction misses}}{\text{Total misses}}$$

We chose this ratio because we want to know how much instructions contribute to the misses in the leader sets. Looking only at the instruction miss count would miss this context. For example, 100 instruction misses out of 200 total misses mean instructions account for half the misses. The same 100 instruction misses out of 2,000 total misses account for only a small share. The ratio helps us distinguish between these cases.

When instructions contribute a larger share of misses, we give them more protection through I-RRIP. When their share is smaller, we use regular RRIP to avoid giving data lower priority unnecessarily. This is a practical way to decide when instruction protection may help, although the ratio alone cannot guarantee a performance gain.

The followers start with I-RRIP. They switch to regular RRIP when the ratio falls below 0.25 and return to I-RRIP when it rises above 0.25. At exactly 0.25, we keep the current policy. We chose this threshold after testing five values, as it gave the highest geometric mean IPC and the lowest load and data MPKI.

We do not reset the counters because we want the policy to follow the dominant behavior observed across the workload so far, rather than react to individual phases. Keeping the accumulated counts makes each decision reflect this overall behavior and reduces the influence of short bursts of misses.

3 SPP with dynamic I-RRIP

3.1 Handling prefetched lines

We identify prefetch requests using `type == PREFETCH` and insert their lines at RRPV 2 in I-RRIP mode, instead of the RRPV 3 used for ordinary data fills. This gives a prefetched line more chance to stay until

the processor needs it. We avoid RRPV 0 because the prediction may be wrong, and an unused line should not receive maximum protection.

Promotion and eviction remain unchanged, so prefetched data can gain protection through hits but can still be evicted. Regular RRIP already inserts all fills at RRPV 2, so it needs no separate prefetch rule.

3.2 Results

Trace	Configuration	IPC	Speedup vs SPP + LRU	Accuracy (%)	Coverage (%)
Trace1	SPP + LRU	0.234036	1.0000×	52.71	8.50
	SPP + dynamic I-RRIP	0.239088	1.0216×	52.45	8.35
Trace2	SPP + LRU	0.252636	1.0000×	41.91	2.28
	SPP + dynamic I-RRIP	0.267078	1.0572×	42.01	2.19
Trace3	SPP + LRU	0.388567	1.0000×	40.11	13.23
	SPP + dynamic I-RRIP	0.391505	1.0076×	40.10	13.22

Table: SPP at the L1D with LRU or prefetch-aware dynamic I-RRIP at the L2C. Speedup is relative to SPP + LRU.

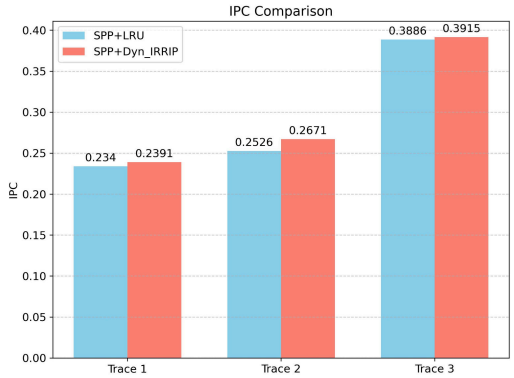


Figure 13: IPC.

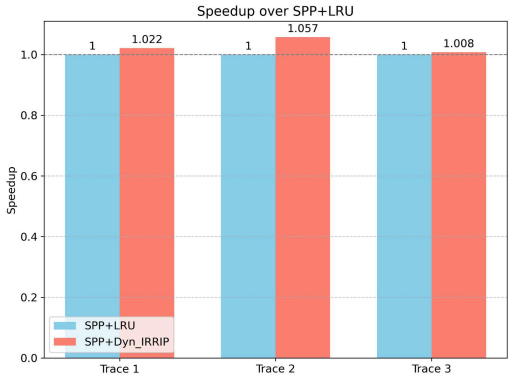


Figure 14: Speedup over SPP + LRU.

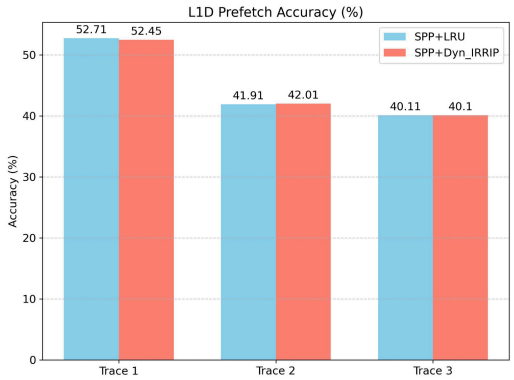


Figure 15: L1D prefetch accuracy.

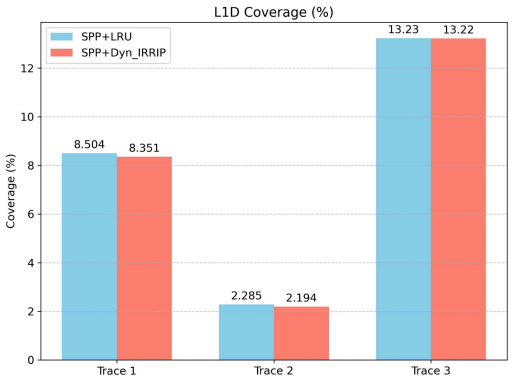


Figure 16: L1D prefetch coverage.

Trace	Configuration	L1D load MPKI	L2C load MPKI	L2 instr. load MPKI	L2 data load MPKI
Trace1	SPP + LRU	9.7700	16.9201	10.3777	6.5424
	SPP + dynamic I-RRIP	9.7863	17.6357	9.1313	8.5044
Trace2	SPP + LRU	13.2286	27.3068	19.0964	8.2104
	SPP + dynamic I-RRIP	13.2409	18.7576	7.9562	10.8014
Trace3	SPP + LRU	2.9533	3.3704	1.1442	2.2262
	SPP + dynamic I-RRIP	2.9534	3.3777	0.8107	2.5670

Table: Load MPKI at the L1D and L2C.

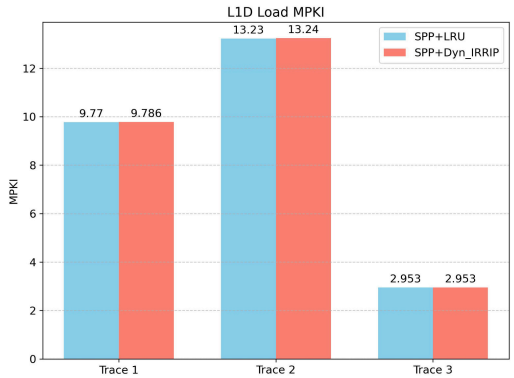


Figure 17: L1D load MPKI.

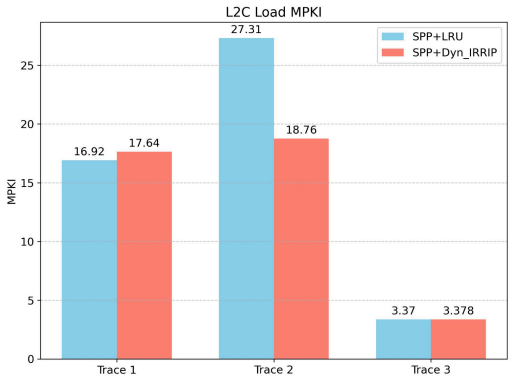


Figure 18: L2C load MPKI.

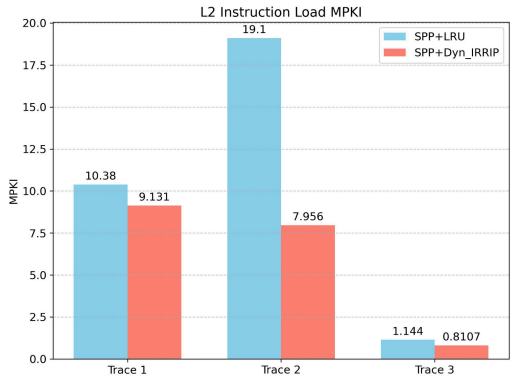


Figure 19: L2C instruction load MPKI.

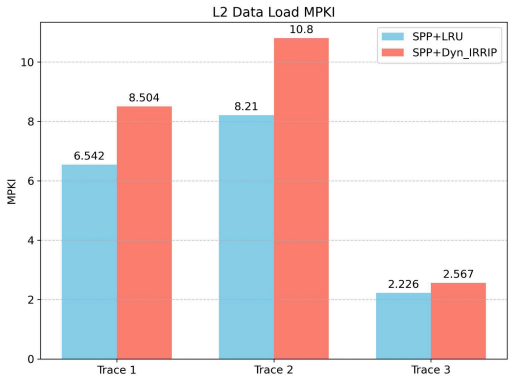


Figure 20: L2C data load MPKI.